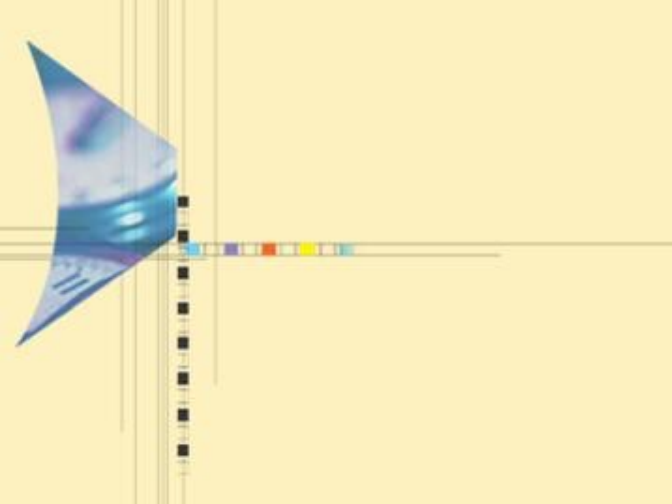


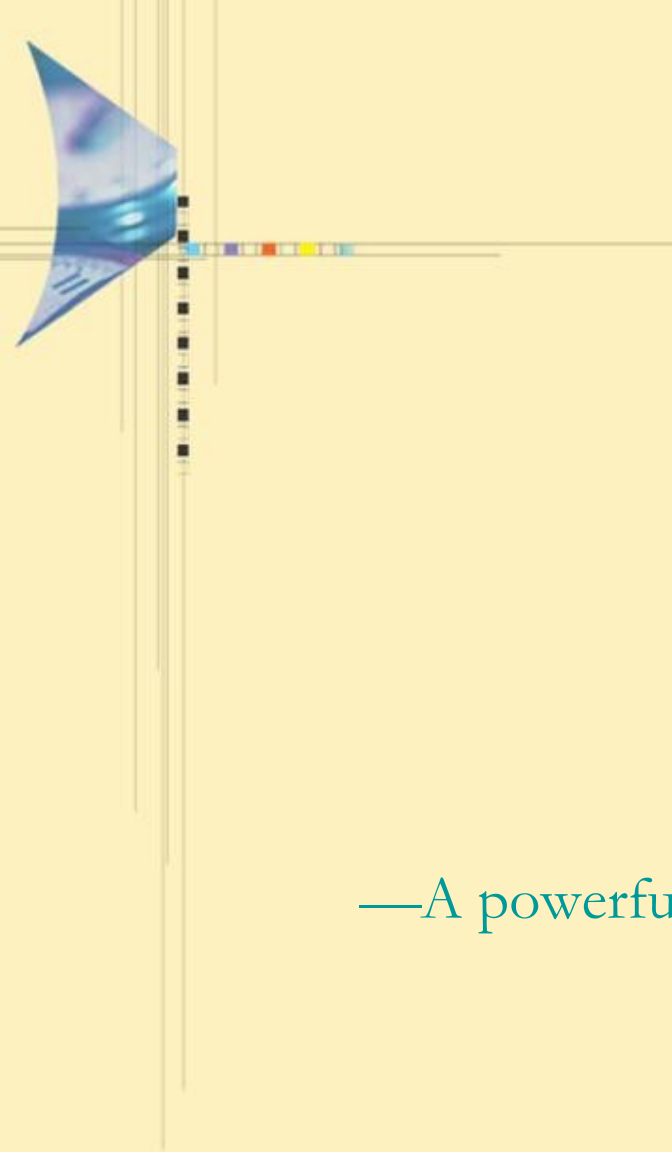


算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）



第四讲

动态规划

—A powerful method for combinatorial problems



主要内容

4.1 引言

4.2 动态规划算法的简单例

4.3 动态规划算法设计框架

4.4 动态规划策略的经典应用

递推式的递归求解与迭代求解方法对比分析

例1: 如下Fibonacci 序列 f_n , 序列中的每个数是它前两个数的和:

$$f_1 = 1, f_2 = 1, f_3 = 2, f_4 = 3, f_5 = 5, f_6 = 8, f_7 = 13, \dots$$

- 该序列的递归定义如下:

$$f(n) = \begin{cases} 1 & \text{if } n = 1 \text{ or } n = 2 \\ f(n-1) + f(n-2) & \text{if } n \geq 3 \end{cases}$$

- 根据定义, 可以得到如下递归求解的简明算法:

```
procedure f(n)
1. if (n=1) or (n=2) then return 1
2. else return f(n-1)+f(n-2)
```



递归求解

- 递推式递归求解（适当展开几步）：

$$\begin{aligned}f(n) &= f(n-1) + f(n-2) \\&= 2f(n-2) + f(n-3) \\&= 3f(n-3) + 2f(n-4)\end{aligned}$$

- 这导致了巨大数量的重复调用。假设计算 $f(1)$ 和 $f(2)$ 需要一个单位时间，那么递归求解过程的时间可以表示为

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \text{ or } n = 2 \\ T(n-1) + T(n-2) & \text{if } n \geq 3 \end{cases}$$

- 递推式的解是： $T(n) = f(n) \approx \Theta(\phi^n)$ $\phi = (1 + \sqrt{5})/2 \approx 1.618\ 03$
- 可以看出，计算 $f(n)$ 所需要的运行时间对于 n 的值是**指数的**。上述递归算法虽然简明，容易理解，也容易差错，但效率非常低

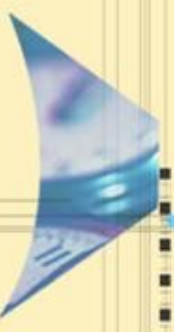


Much more efficient solution?



迭代求解

```
procedure F(n)
1. F1  $\leftarrow$  1
2. F2  $\leftarrow$  1
3. for I  $\leftarrow$  1 to n
4.   F  $\leftarrow$  F1+F2
5.   F1  $\leftarrow$  F2
6.   F2  $\leftarrow$  F
7. end for
8. return F
```



迭代求解

- 采取自底向上的方式递推求值，并把中间结果存储起来以便以后用来计算所要求的解。
- 对于Fibonacci 函数来说，如果从 f_1 开始自底向上地计算直至到 f_n ，只需要 $\Theta(n)$ 时间和 $\Theta(1)$ 空间。和上面的方法相比，可以很大程度地降低时间复杂度。

递推式的递归求解与迭代求解方法对比分析

例2: 二项式系数定义如下:

$$\binom{n}{k} = \begin{cases} 1 & \text{if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{if } 0 < k < n \end{cases}$$

- 与Fibonacci 系数类似, 递归算法的时间复杂度正比于函数自身:

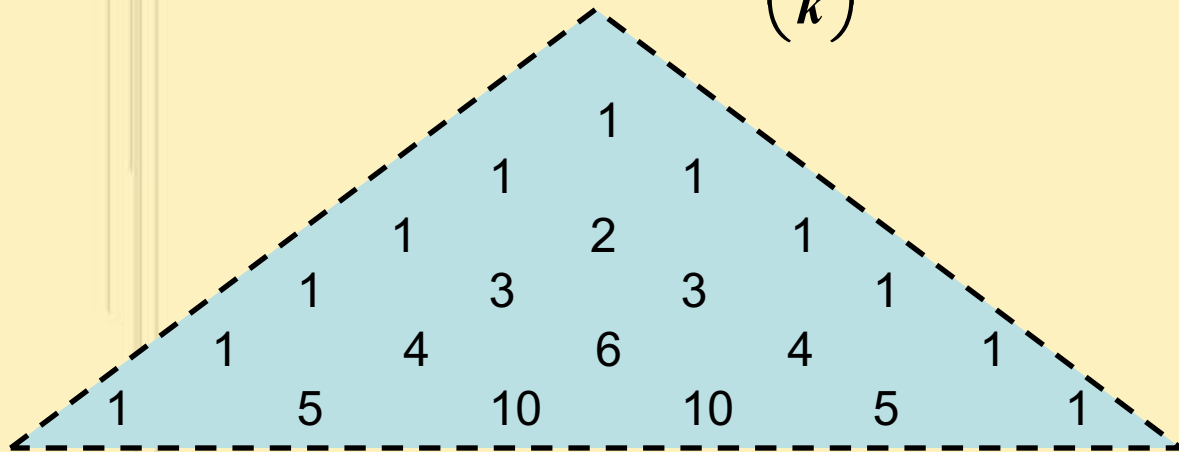
$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

此函数增长迅速. 例如, 当 $k=n/2$ (假设 n 是偶数), 可知:

$$\binom{n}{n/2} = \frac{n!}{((n/2)!)^2} \approx \frac{\sqrt{2\pi n} n^n / e^n}{\pi n (n/2)^n / e^n} \geq \frac{2^n}{\sqrt{\pi n}}$$

迭代求解

- 一种有效计算 $\binom{n}{k}$ 的方法可以用按行构造帕斯卡三角来进行，一旦计算出 $\binom{n}{k}$ 的值，计算就立即停止。

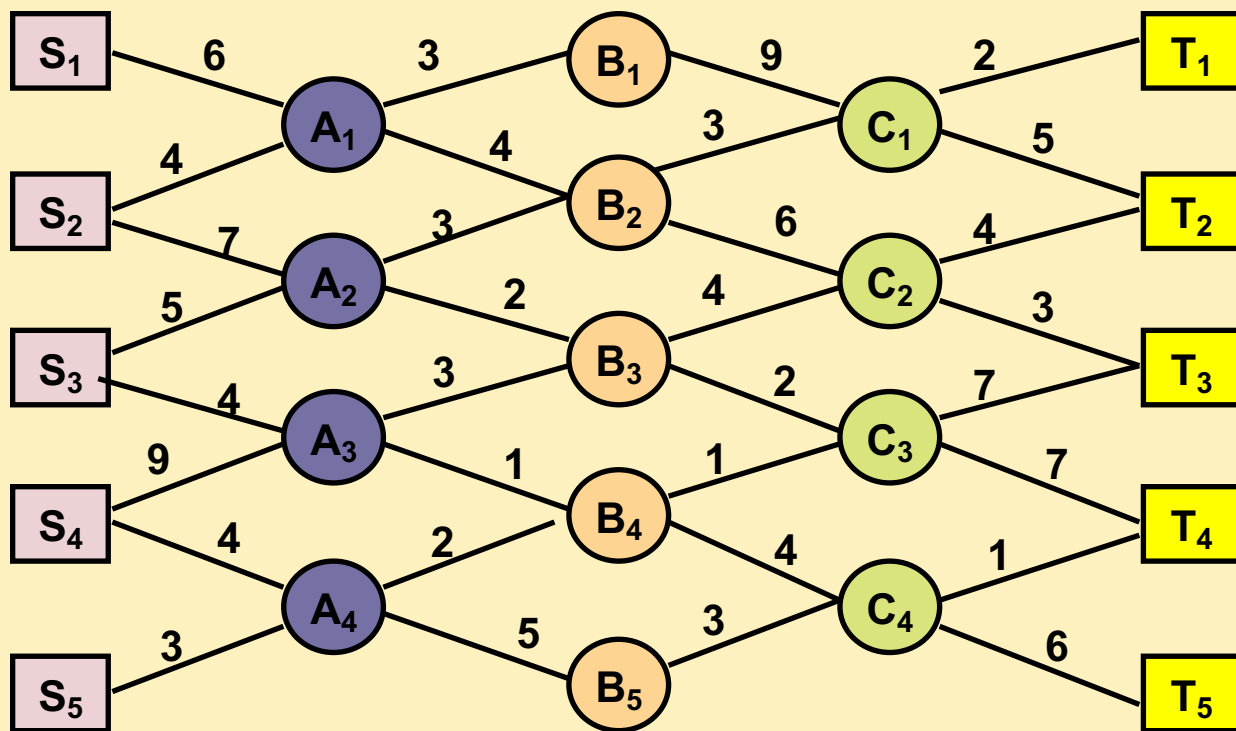


帕斯卡三角形前6行

- 1、每个数等于它上方两数之和。
- 2、每行数字左右对称，由1开始逐渐变大。
- 3、第n行的数字有n项。
- 4、第n行数字和为 2^{n-1} 。

利用这种自下而上迭代的方式，并将后续需要的中间结果储存起来的技术可以设计出特别有效的算法来解决许多组合最优化问题。

例3: 多起点多终点的最短路径



在所有起点到终点的路径($S_i-A_j-B_k-C_l-T_m$)中找一条长度最短的路径



蛮力求解

思想：穷举每一个起点到每一个终点的所有可能的路径，然后计算每条路径的长度，从中找出最短路径。

分析：每条路径由4条边构成，除了最上和最下边的某些结点外，处于中间的结点都有2条边可选。因此，从起点出发的路径条数为 $O(m2^n)$ ，其中， m 为起点个数， n 为每条路径中边的条数，也就是路网的层数。

大量子路径重复计算。。。

更快的方法

- 思想：从终点向起点往回推，把求解过程分为四步。每步对应的子问题终点不变，起点逐步前移，使得前步已经求解的问题是后面新问题的子问题，一直到最后一步求解的问题正好是原问题。

$$F(C_l) = \min_m \{C_l T_m\}$$

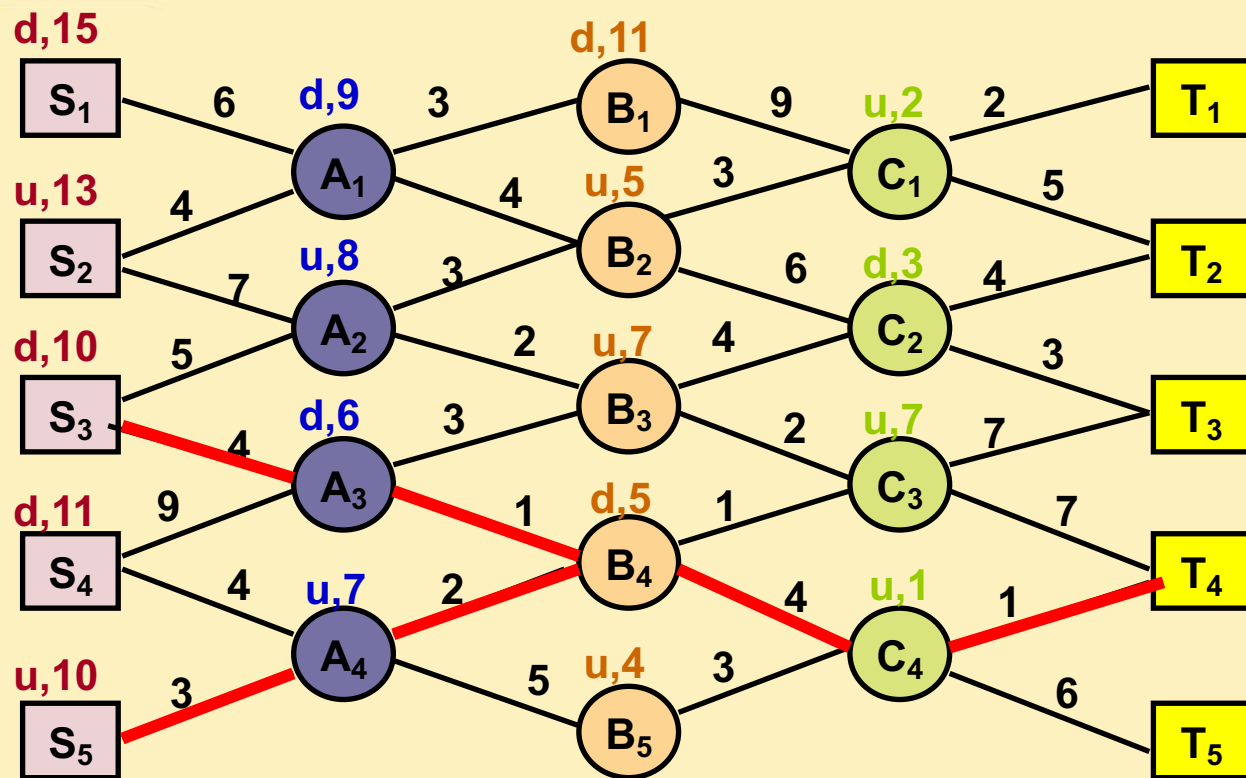
$$F(B_k) = \min_l \{B_k C_l + F(C_l)\}$$

$$F(A_j) = \min_k \{A_j B_k + F(B_k)\}$$

$$F(S_i) = \min_j \{S_i A_j + F(A_j)\}$$

- 任何最短路径的子路径都是相对于子路径的始点和终点的最短路径
- 为找一条最短路径只需从 T_j 开始进行多步判断
- 每次只考虑由前面子问题最优解可能的延伸结果，从而把不可能成为最优解的部分路径尽早删除，提高效率。
- 除终点外，每个结点只需作2次加法（2个分支）1次比较。算法时间复杂度为 $O(mn)$ ，其中 m 代表每层结点数， n 代表层数。

多起点多终点的最短路径



结点标记： C_1 的标记 $u, 2$ 表示从 C_1 向上走到 T_1 ； C_2 的标记 $d, 3$ 表示从 C_2 向下走到 T_3

动态规划技术

- **动态规划**是一种解决多阶段决策问题的优化方法，把多阶段过程转化为一系列单阶段问题，利用各阶段之间的关系，逐个求解。
- 动态规划技术通常用来设计特别有效的算法来**解决组合优化问题**，也可以**改善蛮力搜索算法**的时间复杂性。
- 使用动态规划技术的算法，不是递归调用自身，但问题的基础通常是用递归函数的形式来说明。
- 如果直接实现递推的结果，会导致不止一次递归调用（重复调用），因此动态规划技术采用自下而上的方式递推求值，并把中间结果储存起来（为了避免重复计算），以便将来用于计算所要求的解。



动态规划和分治策略的异同点

共同点:

- 动态规划与分治方法相似，都是基于递归的思想。
- 都将待求解问题分解成若干个子问题。求解子问题，通过合并子问题的解来求解整体问题。



动态规划和分治策略的异同点

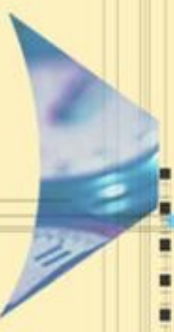
区别:

- **分治法**是把大问题分解成一些**相互独立的**子问题，**递归**的求解这些子问题然后将他们**合并**来得到整个问题的解。当**子问题不独立**时，即当子问题共享子问题时，如果用分治法求解，有些**子问题被重复计算了**许多次。甚至出现虽然不同子问题的数目只有多项式量级时，但由于大量重复计算，计算时间时指数时间复杂度。
- 当子问题不独立时，如果能够**保存已解决的子问题的答案**，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。**而动态规划算法只解决每个子问题一次，然后将其答案保存起来**，从而避免了每次遇到子问题时重新计算答案的工作。即动态规划通过空间开销来节省由于重复计算带来的时间开销。

Those who cannot remember the past are doomed to repeat it.

-----George Santayana, (1905)





最优化问题

- 动态规划通常应用于求解优化问题。
- 最优化问题：在这些问题中，可能有许多可能的解。每个解都有一个值，希望找到一个具有最优（最小或最大）值的解。称这样的解为问题的一个最优解，而不是唯一最优解，因为可能有几个解都能达到最优值。这样的问题称为最优化问题。
- 动态规划技术本质上是对通过自底向上迭代的方式求最优解具有的最优值。

最优化问题

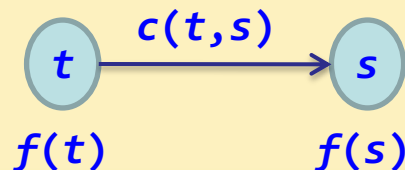
最优性原理：如果问题的最优解所包含的子问题的解也是子问题的最优解，就称该问题具有最优子结构，即满足最优性原理（优化原则）。

$$f(A) = 0$$

$$f(s) = \underset{\text{存在 } \langle t, s \rangle \text{ 的有向边}}{\text{MIN}} \{f(t) + c(t, s)\}$$

$f(s)$ 表示初始状态A到状态s的最短路径长度

子问题的解也是最优的

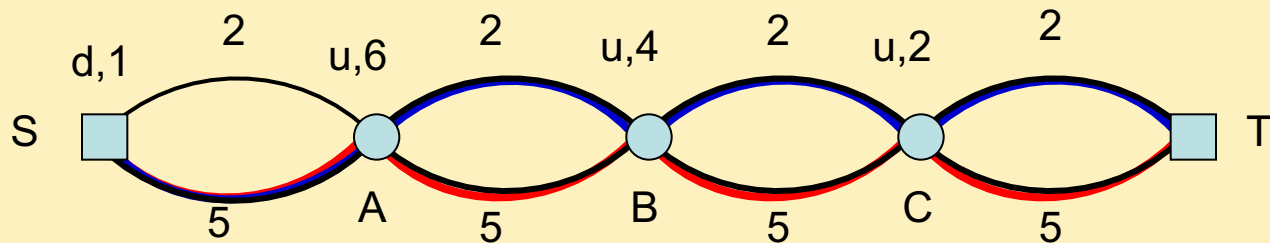


动态规划技术适用条件

- 一个最优决策序列的任何子序列本身一定是相对于子序列的初始和结束状态的最优的决策序列。正是由于这个优化原则，在考虑后面较大子问题最优解时，只需要考虑它的子问题的最优解延伸的结果。

并不是所有优化问题都适用优化原则，有时对某个子问题的解不一定最优，但延伸成整个问题的解时反而最优。

例：求从S到T总长模10的最小路径



最优解：下、下、下、下

动态规划求解（逆向）：下、上、上、上

不满足优化原则，不能使用动态规划设计技术求最优解



主要内容

4.1 引言

4.2 动态规划算法的简单例

4.3 动态规划算法设计框架

4.4 动态规划策略的经典应用

4.2.1 最长公共子序列问题

1、子序列 (subsequence):

给定两个序列: $X=\{x_1, x_2, \dots, x_m\}$, $Z=\{z_1, z_2, \dots, z_k\}$ 。如果存在一个严格递增的X序列的下标序列 $\{i_1, i_2, \dots, i_k\}$ 使得对于所有 $j=1, 2, \dots, k$, 都有 $z_j=x_{i_j}$ 。则称Z是X的**子序列**。

• **例如**, 序列 $Z=\{B, C, D, B\}$ 是序列 $X=\{A, B, C, B, D, A, B\}$ 的子序列, 相应的X序列的递增下标序列为 $\{2, 3, 5, 7\}$ 。

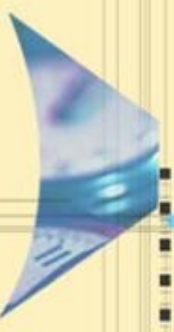
2、公共子序列 (common subsequence)

给定两个序列X和Y, 当另一序列Z既是X的子序列又是Y的子序列时, 称Z是序列X和Y的**公共子序列**。

• **例如**, 设 $X=\{A, B, C, B, D, A, B\}$, $Y=\{B, D, C, A, B, A\}$, 序列 $\{B, C, A\}$ 是X和Y的公共子序列; 序列 $\{B, C, B, A\}$ 也是X和Y的公共子序列。

4.2.1 最长公共子序列问题

- 设在字母表 $\Sigma = \{x, y, z\}$ 上的两个序列： $X = \text{“zxyxyz”}$ 和 $Y = \text{“xyyzx”}$ ，那么序列 “xyy” 同时是 X 和 Y 的长度为 3 的子序列，因此是 X 和 Y 的公共子序列。然而，它不是 X 和 Y 最长的公共子序列，因为长度为 4 的序列 “xyyz” ，也是 X 和 Y 的公共子序列。
- 另外，由于 X 和 Y 这两个序列的公共子序列中没有比 4 更长的公共子序列， X 和 Y 的最长的公共子序列的长度是 4，序列 “xyyz” 是 X 和 Y 的最长公共子序列。



4.2.1 最长公共子序列问题

问题描述

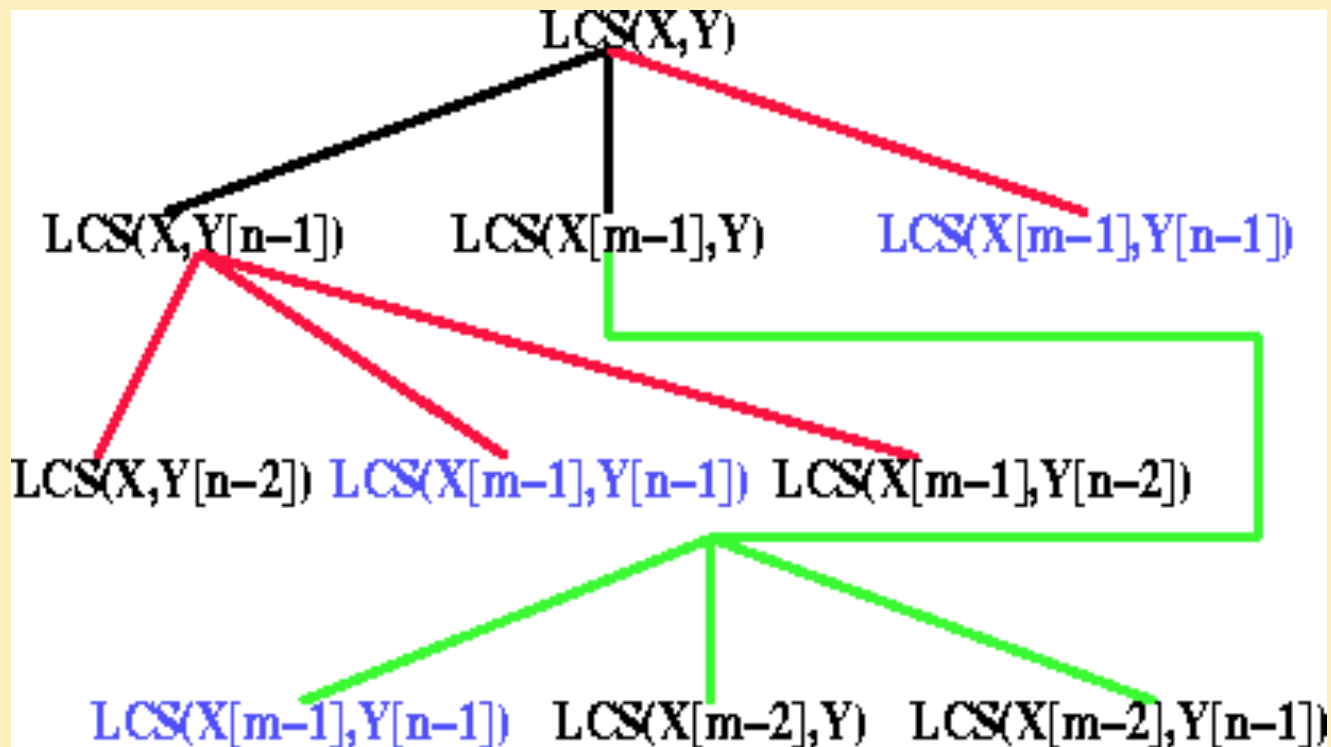
- 最长公共子序列问题 (**longest-common-subsequence (LCS) problem**) : 给定字母表 Σ 上的两个序列X和Y, 求X和Y中的最长公共子序列。.



蛮力搜索算法

- 设两序列分别为： $X = x_1, x_2, \dots, x_m$; $Y = y_1, y_2, \dots, y_n$
- 枚举X所有的 2^m 个子序列, 检查每个子序列是否也是Y的子序列。
- 显然, 对于X的每一个子序列, 在 $\Theta(n)$ 时间内来确定它是否也是Y的子序列。
- 因此, 此蛮力算法的时间复杂性是 $\Theta(n2^m)$, 指数复杂性的。

大量重复的子问题计算



- 不重叠的子问题个数只有 $m \times n$ 个

LCS满足最优子结构性质

第*i*前缀定义:

给定一个序列 $X = x_1, x_2, \dots, x_m$, 定义 $X_i = x_1, x_2, \dots, x_i$ 为 X 的第*i*前缀, $i = 0, 1, \dots, m$.

- 例: 设 $X = A, B, C, B, D, A, B$, 则有: $X_4 = A, B, C, B$, X_0 位是空序列.

观察结论4.1:

令 $X = x_1, x_2, \dots, x_m$, $Y = y_1, y_2, \dots, y_n$ 是两个序列, 令 $Z = z_1, z_2, \dots, z_k$ 是 X 和 Y 的任意一个LCS:

1. 如果 $x_m = y_n$, 则 $z_k = x_m = y_n$ 并且 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的一个LCS;
2. 如果 $x_m \neq y_n$, 则 $z_k \neq x_m$ 蕴含 Z 是 X_{m-1} 和 Y 的一个LCS;
3. 如果 $x_m \neq y_n$, 则 $z_k \neq y_n$ 蕴含 Z 是 X 和 Y_{n-1} 的一个LCS.

观察结论 4.1 的证明

(1) 如果 $x_m = y_n$, 则 $z_k = x_m = y_n$ 并且 Z_{k-1} 是 X_{m-1} and Y_{n-1} 的一个 LCS

反正法:

如果 $z_k \neq x_m$, 则将 $x_m = y_n$ 添加到 Z 的末尾可以得到 X 和 Y 的长度为 $k+1$ 的公共子序列, 这与 Z 是 X 和 Y 的最长公共子序列矛盾。因此 $z_k = x_m = y_n$.

显然, Z_{k-1} 是长度为 $(k-1)$ 的公共子序列 X_{m-1} 和 Y_{n-1} . 现证明它是 X_{m-1} 和 Y_{n-1} 的一个 LCS。如果不是, 则一定存在一个长度大于 $k-1$ 的 X_{m-1} 和 Y_{n-1} 的公共子序列 W . 那么将 $x_m = y_n$ 添加到 W 序列的末尾会得到一个长度大于 k 的 X 和 Y 的公共子序列, 这与 Z 是 X 和 Y 的最长公共子序列矛盾。

观察结论 4.1 的证明

(2) 如果 $x_m \neq y_n$, 则 $z_k \neq x_m$ 蕴含 Z 是 X_{m-1} 和 Y 的一个 LCS

如果 $z_k \neq x_m$, 则 Z 是 X_{m-1} 和 Y 的一个公共子序列。假设 X_{m-1} 和 Y 存在一个长度超过 k 的公共子序列 W , 显然 W 也是 X_m 和 Y 一个公共子序列, 这与 Z 是 X 和 Y 的 LCS 矛盾。

(3) 如果 $x_m \neq y_n$, 则 $z_k \neq y_n$ 蕴含 Z 是 X 和 Y_{n-1} 的一个 LCS

证明同 (2) 类似, 略

由观察结论 4.1 可知, 两个序列的最长公共子序列的前缀序列是这两个序列的前缀序列的一个最长公共子序列。因此, 最长公共子序列问题具有最优子结构性质。

动态规划的求解方法

令 $X = x_1, x_2, \dots, x_m$, $Y = y_1, y_2, \dots, y_n$ 是两个序列

- 如果 $x_m = y_n$, 则首先要找 X_{m-1} 和 Y_{n-1} 的一个LCS, 然后将 $x_m = y_n$ 添加到这个LCS 序列末尾, 得到 X 和 Y 的一个LCS
- 如果 $x_m \neq y_n$, 则先求解如下两个子问题:
 - 找 X_{m-1} 和 Y 的一个LCS
 - 找 X 和 Y_{n-1} 的一个LCS

两个子问题得到的LCS中更长的那个就是 X 和 Y 的LCS

- 令 $c[i, j]$ 储存序列 X_i 和 Y_j 的LCS的长度。当 $i = 0$ 或者 $j = 0$ 时, 表示其中对应的序列长度为0, LCS长度为0;
- 令 $b[i, j]$ 记录同 $c[i, j]$ 对应的求 X_i 和 Y_j 的LCS Z_k 时的最优决策。

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max(c[i, j-1], c[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

算法4.1 LCS

输入:字母表 Σ 上的两个字符串X和Y, 长度分别为n和n

输出: X和Y最长公共子序列的长度。

1. **for** $i \leftarrow 0$ **to** m // $j=0$ 时, $c[i,j]=0$
2. $c[i,0] \leftarrow 0$
3. **end for**
4. **for** $j \leftarrow 0$ **to** n // $i=0$ 时, $c[i,j]=0$
5. $c[0,j] \leftarrow 0$
6. **end for**
7. **for** $i \leftarrow 0$ **to** m
8. **for** $j \leftarrow 0$ **to** n
9. **if** $x_i = y_j$ **then** $c[i,j] \leftarrow c[i-1,j-1] + 1$
10. **else** $c[i,j] \leftarrow \max\{c[i,j-1], c[i-1,j]\}$
11. **end if**
12. **end for**
13. **end for**
14. **return** $c[m,n]$

例: $X = A, B, C, B, D, A, B$; $Y = B, D, C, A, B, A$

		j	0	1	2	3	4	5	6
		y_j		B	D	C	A	B	A
0	x_i		0	0	0	0	0	0	0
1	A		0	↑	↑	↑	↖1	←1	↖1
2	B		0	↖1	←1	←1	↑1	↖2	←2
3	C		0	↑1	↑1	↖2	←2	↑2	↑2
4	B		0	↖1	↑1	↑2	↑2	↖3	←3
5	D		0	↑1	↖2	↑2	↑2	↑3	↑3
6	A		0	↑1	↑2	↑2	↖3	↑3	↖4
7	B		0	↖1	↑2	↑2	↑3	↖4	↑4

同时求LCS的长度和LCS序列本身

算法 4.2 LCS1

输入: 字母表 Σ 上的两个字符串X和Y, 长度分别为n和n

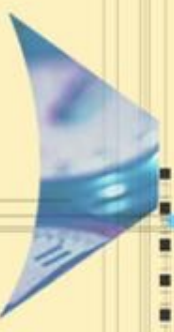
输出: 矩阵c和b, 分别存储X和Y的LCS长度和对应的最优决策

```
1.  for  $i \leftarrow 0$  to  $m$ 
2.       $c[i, 0] \leftarrow 0$ 
3.  end for
4.  for  $j \leftarrow 0$  to  $n$ 
5.       $c[0, j] \leftarrow 0$ 
6.  end for
7.  for  $i \leftarrow 1$  to  $m$ 
8.      for  $j \leftarrow 1$  to  $n$ 
9.          if  $x_i = y_j$  then  $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ ,  $b[i, j] \leftarrow "\diagdown"$ 
10.         else if  $c[i - 1, j] \geq c[i, j - 1]$  then  $c[i, j] \leftarrow c[i - 1, j]$ ,  $b[i, j] \leftarrow "\uparrow"$ 
11.         else  $c[i, j] \leftarrow c[i, j - 1]$ ,  $b[i, j] \leftarrow " \leftarrow "$ 
12.         end if
13.     end if
14. end for
15. end for
16. return c and b
```

基于矩阵b, 追踪构造LCS

算法4.3 PRINT-LCS(b, X, i, j)

1. **if** $i = 0$ or $j = 0$ **then return**
2. **end if**
3. **if** $b[i, j] = "\diagdown"$ **then** PRINT-LCS($b, X, i - 1, j - 1$), **print** x_i
4. **else if** $b[i, j] = "\uparrow"$ **then** PRINT-LCS($b, X, i - 1, j$)
5. **else** PRINT-LCS($b, X, i, j - 1$)
6. **end if**



算法分析

如果只需要计算最长公共子序列的长度，则算法的空间需求可大大减少。事实上，在计算 $c[i][j]$ 时，只用到数组 c 的第 i 行和第 $i-1$ 行。因此，用2行的数组空间就可以计算出最长公共子序列的长度。进一步的分析还可将空间需求减至 $O(\min(m,n))$ 。

定理4.1 最长公共子序列问题的最优解能够在 $\Theta(nm)$ 时间和 $\Theta(\min\{m,n\})$ 空间内得到。

4.2.2 矩阵链乘法

问题描述:

- 给定 n 个矩阵 $A_1, A_2, \dots, A_n$ 链（序列），其中 A_i 与 A_{i+1} 是可乘的， $i=1, 2, \dots, n-1$ 。计算这 n 个矩阵的连乘积 $A_1 \times A_2 \times \dots \times A_n$ 。
- 由于矩阵乘法满足结合律，故计算矩阵链的连乘积可以有許多不同的计算次序。这种计算次序可以用加括号的方式来确定。若一个矩阵连乘积的计算次序完全确定，也就是说该连乘积已完全加括号，则可以依此次序反复调用2个矩阵相乘的标准算法计算出矩阵连乘积。
- 例如，矩阵链 A_1, A_2, A_3, A_4 的连乘积有5种不同的完全加括号方式，代表5种不同的计算次序：
 - $(A_1 (A_2 (A_3 A_4)))$,
 - $(A_1 ((A_2 A_3) A_4))$,
 - $((A_1 A_2) (A_3 A_4))$,
 - $((A_1 (A_2 A_3)) A_4)$,
 - $((((A_1 A_2) A_3) A_4))$.

4.2.2 矩阵链乘法

- 每一种完全加括号的方式对应于一个矩阵连乘积的计算次序，这决定着作乘积所需要的计算量。
- 首先考虑两个矩阵相乘的成本。标准的算法由下面的伪代码给出：

```
Algorithm MATRIX-MULTIPLY(A, B)
if  $columns[A] \neq rows[B]$ 
then error "incompatible dimensions"
else for  $i \leftarrow 1$  to  $rows[A]$  //p
    for  $j \leftarrow 1$  to  $columns[B]$  //r
         $C[i, j] \leftarrow 0$ 
        for  $k \leftarrow 1$  to  $columns[A]$  //q
             $C[i, j] \leftarrow C[i, j] + A[i, k] \cdot B[k, j]$ 
        end for
    end for
end for
end if
return C
```

相乘的两个矩阵必须是兼容的
(**compatible**) :即A矩阵的列数必须等于B矩阵的行. 若A是一个 $p \times q$ 矩阵, B是一个 $q \times r$ 矩阵, 则计算其乘积 $C=AB$ 的标准算法中, 需要进行 pqr 次数乘。

4.2.2 矩阵链乘法

- 先考察3个矩阵 $\{A_1, A_2, A_3\}$ 连乘的情况。设这三个矩阵的维数分别为 10×100 , 100×5 , and 5×50 。

加括号方式	第一次相乘次数	第二次相乘次数	总相乘次数
$((A_1 A_2) A_3)$	$10 \cdot 100 \cdot 5 = 5000$	$10 \cdot 5 \cdot 50 = 2500$	7500
$(A_1 (A_2 A_3))$	$100 \cdot 5 \cdot 50 = 25000$	$10 \cdot 100 \cdot 50 = 50000$	75000

如何确定计算矩阵链 $A_1, A_2, \dots, A_n$ 连乘积的计算次序（完全加括号方式），使得依此次序计算矩阵连乘积需要的数乘次数最少？

蛮力方法：穷举所有完全加括号方式

- 对于 n 个矩阵的连乘积，设其不同的计算次序为 $P(n)$
- 当 $n = 1$ ，只有一个矩阵，因此 $P(n)=1$
- 当 $n \geq 2$ ，假设连乘积是由前 k 个矩阵的连乘子积和后 $n-k$ 个矩阵的连乘子积相乘得到。对于前 k 个矩阵，一共有 $P(k)$ 种方法放置括号。对于 $P(k)$ 中的每一种放置括号方法，余下 $n-k$ 个矩阵都有 $P(n-k)$ 个放置括号方法，因此总共有 $P(k)P(n-k)$ 种放置括号方法。由于 k 可以是1到 $n-1$ 中的任意值，对于 n 个矩阵放置括号的所有方法数由下面的和式给出：

$$P(n) = \begin{cases} 1 & \text{if } n = 1. \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2. \end{cases}$$

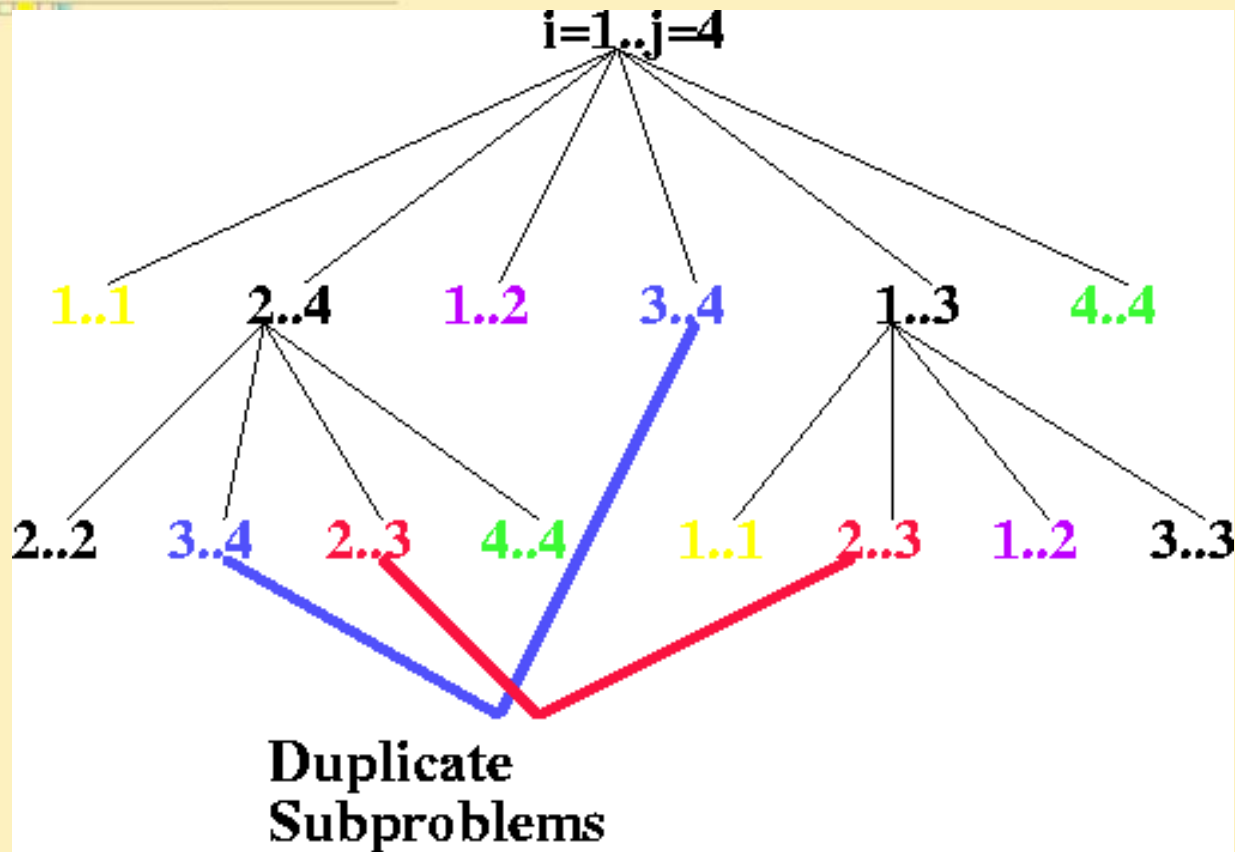
蛮力方法：穷举所有完全加括号方式

- 上述递推式的解：

$$P(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{(2n-2)!}{n((n-1)!)^2} \approx \frac{4^n}{4\sqrt{\pi n^{1.5}}}$$

- 由于对于每个括弧号表达式，找到数量乘法次数的时间耗费是 $\Theta(n)$ ，这样用蛮力方法找到 n 个矩阵相乘的最优方法所需的运行时间是 $\Omega(4^n/n^{0.5})$ ，它甚至对于一个中等规模的 n 值也是不切实际的。

大量重复子问题计算



只有关于 n 的多项式个数的不同子问题个数: $\binom{n}{2} + 2 = \Theta(n^2)$

矩阵链乘法问题满足最优子结构性质

- 假定最优的计算次序是在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开，生成两个子矩阵链 $A_1 \dots A_k$ 和 $A_{k+1} \dots A_n$ 。则 $A_1 \dots A_n$ 的最优连乘积计算次序所包含的给矩阵子链 $A_1 \dots A_k$ 和 $A_{k+1} \dots A_n$ 的连乘积计算次序也是最优的。
- 矩阵子链的所耗费的计算量越低，则矩阵链 $A_1 \dots A_n$ 所耗费的计算量越低
- 总共的计算量包括 $\text{cost}(A_1 \dots A_k) + \text{cost}(A_{k+1} \dots A_n) + \text{cost}$ （两个矩阵子链的乘积生成的矩阵相乘）。而无论子问题的解决方案如何，最后一项的计算量不变。因此，总计算量最小，必须有 $\text{cost}(A_1 \dots A_k)$ 和 $\text{cost}(A_{k+1} \dots A_n)$ 都最小。

动态规划的求解方法

- 将矩阵连乘积 $A_i A_{i+1} \dots A_j$ 简记为 $A_{i..j}$, $i \leq j$;
- 设数组 $p[0..n]$ 存放矩阵链中各矩阵的维度, A_i 的维度为 $p[i-1] \times p[i]$.
- 设计算 $A_{i..j}$ ($1 \leq i \leq j \leq n$)所需要的最少数乘次数为 $m[i,j]$, 则原问题的最优值为 $m[1,n]$
 - 当 $i = j$ 时, $m[i,i] = 0$ (单个矩阵) , $i = 1, 2, \dots, n$
 - 当 $i < j$ 时, 假设最优的矩阵链划分点在 A_k 和 A_{k+1} ($i \leq k < j$)之间, 则有 $m[i,j] = m[i,k] + m[k+1,j] + p[i-1]p[k]p[j]$
 - 由于事先并不知道最优的划分点, 因此需要尝试 k 的所有 $j-i$ 种可能取值。因此递归定义 $m[i,j]$ 如下:

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p[i-1]p[k]p[j]\} & \text{if } i < j \end{cases}$$

动态规划求解方法

- 设 n 个矩阵的矩阵链 $A_1, A_2, \dots, A_n$, 用长度为 $n+1$ 的数组 $p[0..n]$ 存放矩阵链中 n 个矩阵的维度, A_i 的维度为 $p[i-1] \times p[i]$. $i = 1, 2, \dots, n$ 。 $p[0]$ 存放的 A_0 行数。
- 辅助表 $m[1..n, 1..n]$ 中 $m[i, j]$ 存储计算 $A_{i..j}$ 所需最小的数乘次数。
- 辅助表 $s[1..n, 1..n]$ 中 $s[i, j]$ 记录 $m[i, j]$ 对应的矩阵链划分点矩阵的索引编号, 基于表 s 构建一个最优的矩阵链的连乘积方案。

算法 4.4 MATRIX-CHAIN-ORDER(p)

输入: $p[0..n]$, n 个矩阵的维度数组。 A_i 的维度为 $p[i-1] \times p[i]$. $i = 1, 2, \dots, n$.

输出: $m[1..n, 1..n]$ and $s[1..n, 1..n]$, 分别存储最小的数乘次数和最优决策.

```
n ← length[p] - 1
for i ← 1 to n
    m[i, i] ← 0
end for
for l ← 2 to n      ▷ l is the chain length.
    for i ← 1 to n - (l - 1)
        j ← i + (l - 1)
        m[i, j] ← ∞
        for k ← i to j - 1
            q ← m[i, k] + m[k + 1, j] + p[i-1] p[k] p[j]
            if q < m[i, j]
                then m[i, j] ← q, s[i, j] ← k
            end if
        end for
    end for
end for
return m and s
```



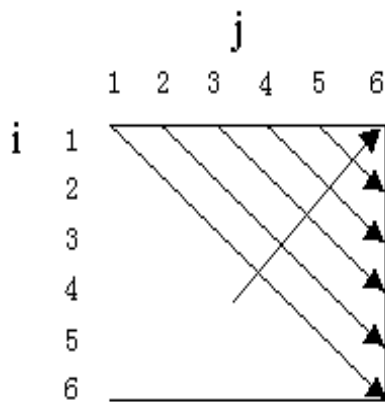
例1

The length of chain is 5, and the dimensions is:
 $A_1 5 \times 10, A_2 10 \times 4, A_3 4 \times 6, A_4 6 \times 10, A_5 10 \times 2$

$m[1,1]=0$	$m[1,2]=200$	$m[1,3]=320$	$m[1,4]=620$	$m[1,5]=348$
	$m[2,2]=0$	$m[2,3]=240$	$m[2,4]=640$	$m[2,5]=248$
		$m[3,3]=0$	$m[3,4]=240$	$m[3,5]=168$
			$m[4,4]=0$	$m[4,5]=120$
				$m[5,5]=0$

例2

A_1	A_2	A_3	A_4	A_5	A_6
30×35	35×15	15×5	5×10	10×20	20×25



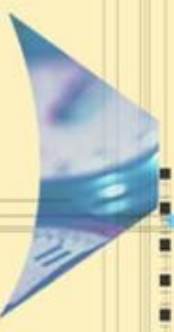
(a) 计算次序

		j					
		1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

(b) $m[i][j]$

		j					
		1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0

(c) $s[i][j]$



算法分析

- 主要计算量取决于算法中对 l , i 和 k 的3重循环。对每重循环最多执行 $n-1$ 次;
- 循环体内的计算量为 $O(1)$;
- 3重循环的总次数为 $O(n^3)$ 。
- 算法的时间复杂性是 $\Omega(n^3)$. 算法的空间复杂度是 $\Theta(n^2)$, 主要是用于存储 m 和 s 表.



构造最优解

算法4.5 PRINT-OPTIMAL-PARENS(s, i, j)

1 **if** $i = j$

2 **then** print " A " i

3 **else** print "("

4 PRINT-OPTIMAL-PARENS($s, i, s[i, j]$)

5 PRINT-OPTIMAL-PARENS($s, s[i, j] + 1, j$)

6 print ")"

求矩阵链的连乘积

算法 4.6 MATRIX-CHAIN-MULTIPLY

Comment: Let $A = A_1, A_2, \dots, A_n$.

1. Call Matrix-Chain-Order
2. Call Matrix-Chain-Multiply

过程 Matrix-Chain-Multiply(A, s, i, j)

if $i < j$

then $x = \text{Matrix-Chain-Multiply}(A, s, i, s[i,j])$

$y = \text{Matrix-Chain-Multiply}(A, s, s[i,j]+1, j)$

return Matrix-Multiply(x, y)

else return A_i

end if



主要内容

4.1 引言

4.2 动态规划算法的简单例

4.3 动态规划算法设计框架

4.4 动态规划策略的经典应用



动态规划策略适用的问题

- 动态规划算法通常用来求解满足优化原则的优化问题。
- 动态规划算法与分治法类似，其设计的基本思想也是将待求解问题分解成若干个同性质的子问题。
- 但是经分解得到的子问题往往不是互相独立的。





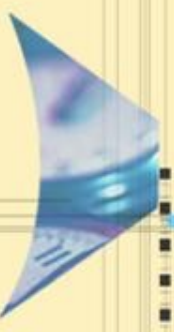
动态规划算法设计的一般步骤

- Step 1:
 - 分析最优解的结构特征，必须满足最优子结构性质；
- Step 2:
 - 递归定义最优值计算公式（动态规划方程）；
- Step 3:
 - 基于Step 2的最优值计算公式，以自底向上的方式计算最优值；
- Step 4:
 - 根据Step 3计算过程中保留的决策信息，构造最优解。



动态规划算法设计的一般步骤

- Steps 1–3 是动态规划求解问题的基础.
- Step 4 在只要求得到问题的最优解的值时, 可以忽略不要.
- 如果需要执行Step 4, 则在step 3 计算最优值的过程中可能需要保留每一步最优决策的额外信息, 以便最终能据此构造一个最优解.



最优子结构-动态规划策略的必要前提

- 问题的最优解中包含的子问题的解也是相对于子问题的最优解。这种性质称为**最优子结构性**质。
- 利用问题的最优子结构性质，保证以自底向上迭代地从子问题的最优解逐步构造出整个问题的最优解。
- 最优子结构是问题能用动态规划算法求解的前提。

重叠子问题-动态规划策略的优势体现

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为**子问题的重叠性质**。
- 动态规划算法，对每一个子问题只解一次，而后将其解保存起来，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。
- 通常不同的子问题个数随问题的大小呈多项式增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。
- 子问题的重叠性质才能体现动态规划策略以空间换时间的优势。

